

```

timescale 1ns / 1ps
`include "parameters.vh"
`default_nettype none

module data_mem (
    input wire clk,
    input wire rst_n,
    input wire i_stb,
    input wire i_wr_en,
    input wire [31:0] i_addr,
    input wire [31:0] i_write_data,
    input wire [4:0] i_amo_op, // Atomic operation selector
    output wire o_rd_ack,
    output reg [31:0] o_read_data
);

reg [31:0] memory[1023:0]; // Byte addressable memory
reg [31:0] reserved_addr; // For LR/SC tracking

assign o_rd_ack = ~rst_n ? 1'b0 : 1'b1;

// Atomic Operation Handling
always @(posedge clk) begin
    if (rst_n) begin
        if (i_stb) begin
            case (i_amo_op)
                `AMO_ADD: memory[i_addr] <= memory[i_addr] + i_write_data;
                `AMO_AND: memory[i_addr] <= memory[i_addr] & i_write_data;
                `AMO_OR:  memory[i_addr] <= memory[i_addr] | i_write_data;
                `AMO_XOR: memory[i_addr] <= memory[i_addr] ^ i_write_data;
                `AMO_MIN: memory[i_addr] <= ($signed(memory[i_addr]) <
$signed(i_write_data)) ? memory[i_addr] : i_write_data;
                `AMO_MAX: memory[i_addr] <= ($signed(memory[i_addr]) >
$signed(i_write_data)) ? memory[i_addr] : i_write_data;
                `AMO_MINU: memory[i_addr] <= (memory[i_addr] < i_write_data) ?
memory[i_addr] : i_write_data;
                `AMO_MAXU: memory[i_addr] <= (memory[i_addr] > i_write_data) ?
memory[i_addr] : i_write_data;
                `AMO_SWAP: memory[i_addr] <= i_write_data;

                // Load-Reserved (LR)
                `AMO_LR: begin
                    o_read_data <= memory[i_addr];
                    reserved_addr <= i_addr;
                end
            endcase
        end
    end
end

```

```

        // Store-Conditional (SC)
        `AMO_SC: begin
            if (reserved_addr == i_addr) begin
                memory[i_addr] <= i_write_data;
                o_read_data <= 32'b0; // SC successful (return 0)
            end else begin
                o_read_data <= 32'b1; // SC failed (return 1)
            end
        end
    end

    default: o_read_data <= 32'b0;
endcase
end
end
end

// Normal Read Operation
always @(*) begin
    if (~rst_n) begin
        o_read_data = 32'd0;
    end else if (i_stb) begin
        o_read_data = memory[i_addr];
    end
end
end

endmodule

```